

Exercise: R functions

Workshop – Introduction to R

1 Importing data

a) Download the excel file **cardata.xlsx** from https://github.com/Swiss-Paraplegic-Research/Workshop-R-Introduction/tree/main/Part2_RFunctions/Exercise and put it in the same folder where you have stored your R script.

b) Although an excel file can be read directly into R, we will first see how to turn it into the simpler “csv” (comma separated values) format to use in R. Open the excel file and turn it into a .csv file by going to “File > Save As > Browse” and choosing “CSV (Comma delimited)” as the format before saving it. Open the newly created .csv file in a simple text editor (e.g. Notepad on Windows) and look at it.

c) Check what your current working directory is.

d) Set the working directory to the folder where the .csv file is located.

e) Read in the .csv file to R by using the **read.csv** function. Depending on the used separator sign you might need to adapt your command (see slides). Store the imported data under the name **cardata** and look at this object, what do you see?

f) As mentioned, we can also read in an excel file directly. To do this we need to first install and load an additional R-package. Install and load the **readxl** package to make its contents available.

g) Now you can read in the excel file using the following command. Look at the resulting data frame.

```
dat <- as.data.frame(read_excel("cardata.xlsx"))
```

2 Ready-to-use functions

a) In this first exercise we will work with the **mpg** variable of the **cardata** data set. It contains the miles per gallon value for each car. The following list contains a selection of functions which can be applied to vectors. Try them out on the **mpg** column.

- **mean** to calculate the mean value
- **sd** to calculate the sample standard deviation and **var** for the sample variance.
- **sum** to add all elements of a vector together.
- **min** and **max** to get the smallest and largest element of a vector. **range** returns these values in one vector.
- **length** to show how many elements a vector contains.

3 Learning about new functions in R

R offers functions for many different tasks. However, often one might not know whether a specific function exists and under which name it is available. And if it exists, one first needs to learn how it is applied. Such information can be found in the internal help pages or by browsing the internet.

- a) Let's look again at the `mpg` variable of the `cardata` data. We want to show the `mpg` vector with all elements sorted by their values in increasing order. Try to find out whether there is a function for that purpose available and if yes, use it.
- b) **Extra:** Look at the help page of that function. Can you find a way to sort the values of `mpg` in descending order instead?
- c) Now we want to calculate the Pearson correlation coefficient between the two columns `mpg` and `disp`. Again, try to find the corresponding R function and apply it.
- d) **Extra:** Can you find a way to calculate the Spearman correlation instead of the Pearson correlation?

4 Writing your own function

- a) Write a function which takes two input arguments (both are expected to be numbers) and calculates the sum of the two.
- b) There is already a function in R to calculate the mean value of a vector. Try to write your own function which takes a (numeric) vector as input and returns its mean value (without using the `mean` function).
- c) **Extra:** Try to write a function which takes a (numeric) vector as an input and scales it to the 0-1 range (linear transformation so that the smallest element is zero and the largest element is 1). To do this, we must first subtract the smallest element from all elements and then divide the elements by the largest element.
- d) **Extra:** Extend the function from the previous exercise so that it not only returns the scaled vector but also the unscaled original vector.

5 sample function in R

There are many functions in R which make use of a random number generator. The `sample` function for example can be used to take a random sample from a vector. Try to guess what the following command does and run it multiple times:

```
sample(1:10, size = 5)
```

The above call takes a random sample of the vector `1:10`. Because it is a *random* sample the result can differ everytime we run the command. In order to make the results of R code including random processes reproducible, we can “fix” the random number generator by setting a “random seed”. For example, run the two commands below multiple times after each other and observe the result:

```
set.seed(4984928)
sample(1:10, size = 5)
```

You should observe that the (random) result stays the same. The number which we put into the `set.seed` function can be chosen arbitrarily, each number fixes the random number generator at a different state.

- a) Try to write a function which takes two input arguments: A data frame and a number. The function should then pick a random sample from the data frame (random selection of rows) of the size specified by the second input argument and return this new data frame containing the subsample.

6 Exporting data

In the previous exercise we have created a function which takes a random subsample from a data frame. Apply the function to our `cardata` data frame to take a random subsample of it. Let's assume that we now want to export this subsample to a file for further storage. We can for example use the `write.csv` function to write a data frame into a .csv file. Use the command below to create a .csv file from our data frame (replace `NAME_OF_DATAFRAME` with the name of your data frame). The file will be saved in the working directory. Look in your working directory if the file was indeed created.

```
write.csv(NAME_OF_DATAFRAME, file = "data_prep.csv", row.names = FALSE)
```